

Entity Definitions

[Previous: Lifecycle Model](#) | [Documentation Home](#) | [Next: External Entity Definition Sources](#)

Coredeux entities are configured through YAML.

The YAML tells the framework how to identify the entity, which persistence implementation to use, and which optional modules are enabled.

Root Structure

```
coredeux:
  entities:
    - full-class-name: com.example.customer.Customer
      name: customer
      identifier: id
      storage:
        data-access-service: postgresCoredeuxJpaDataAccessService
      modules:
        - name: validators
          enabled: true
          handlers:
            - customerRequiredFieldsValidator
        - name: hooks
          enabled: true
          handlers:
            - customerLifecycleHook
        - name: audit
          enabled: true
          handlers:
            - defaultAuditHandler
      config:
        operations:
          - SAVE
          - UPDATE
          - DELETE
```

Core Entity Fields

The current core fields are:

- `full-class-name`
- `name`
- `identifier`
- `storage`

full-class-name

Fully qualified Java class name of the entity.

This field is mandatory.

name

Logical framework name for the entity.

If omitted, the loader defaults it to the full class name.

identifier

Field name used by the framework to resolve the entity identifier through reflection.

This field is mandatory.

storage

Storage block for persistence configuration.

Current required field inside `storage`:

- `data-access-service`

Example:

```
storage:  
  data-access-service: defaultCoredeuxJpaDataAccessService
```

Module List

Additional functionality is modeled through the entity's `modules` list.

Each module entry can contain:

- `name`
- `enabled`
- `handlers`
- `config`

name

Logical module name such as:

- `validators`
- `hooks`
- `audit`

enabled

Whether the module is active for the entity.

handlers

List of Spring bean names to execute for that module.

config

Module-specific configuration object.

The structure depends on the module type.

Special Module Configs

Most module configuration is stored as immutable maps and lists.

One current special case is the `attributes` module, which is converted into a typed attribute model when it appears in the YAML.

That keeps the general loader simple while still allowing typed structures where the framework needs them.

Audit Module Example

```
- name: audit
  enabled: true
  handlers:
    - defaultAuditHandler
  config:
    operations:
      - SAVE
      - UPDATE
      - DELETE
```

Data Access Routing

The `storage.data-access-service` field tells the framework which `CoredeuxDataAccessService` bean should handle persistence for the entity.

That allows different entities to use different adapters inside the same application.

Examples:

- generic JPA entity
- PostgreSQL JSONB-aware JPA entity
- JDBC-backed entity
- MongoDB-backed entity
- Elasticsearch-backed entity
- Redis-backed entity

Validation Rules

Current loader rules:

- `full-class-name` is required
- `identifier` is required
- `storage.data-access-service` is required
- `name` defaults to `fullClassName` when omitted
- module fields are optional unless required by the specific module handler

Design Guidance

Only stable cross-framework entity information should remain top-level.

Feature-specific behavior should stay under `modules` so future modules can be added without repeatedly changing the core entity contract.

[Previous: Lifecycle Model](#) | [Documentation Home](#) | [Next: External Entity Definition Sources](#)